Relatório técnico — Pedro, assistente de compras

Curso: Agentes Cognitivos com LangChain (Infnet). **Atualizado:** 2026-11-11. **Modelo:** gpt-4.1-mini. **Testes automáticos:** 143, todos passando.

Este relatório explica como o projeto responde às 29 rubricas do guia integrado (R01–R29). Cada rubrica tem aqui um resumo; a resposta completa, com suas tabelas e seus limites, está em docs/respostas_rubricas.md. As execuções com o modelo real estão salvas em docs/rastros/ e o registro do que foi executado, em docs/evidencias.md.

1. Resumo

Pedro é um assistente que ajuda a comprar peças de reposição de eletrodomésticos: compara as ofertas de vários fornecedores para uma peça e uma quantidade, e recomenda uma. Todos os dados são sintéticos.

- **O que faz bem.** Compara fornecedores com regras exatas, mantém uma conversa de vários turnos, responde perguntas sobre políticas com a fonte, confirma um CEP com uma API real e não publica nada sem verificar.
- **Resultado medido.** Na avaliação vigente no LangSmith, feita em português, acerta o vencedor e o valor em 14 de 14 recomendações, e as 24 propostas do modelo passam na verificação na primeira tentativa.
- **Como se trabalhou.** Cada rubrica tem uma execução salva. Várias execuções com o modelo real revelaram falhas que os testes não viam; elas foram diagnosticadas, corrigidas e medidas de novo, e o antes e o depois estão guardados.
- **Migração para português.** Em 2026-11-11 o projeto inteiro foi migrado do espanhol para o português (o glossário de nomes está em docs/glossario_migracao.md). As regras de negócio e os preços do catálogo não mudaram. Todas as demonstrações e a avaliação no LangSmith foram reexecutadas em português, com os mesmos números de qualidade. Só a pergunta do diagnóstico de recuperação T24 precisou ser medida de novo (ver R23).
- **O que fica em aberto.** O vídeo de demonstração e três suposições que o professor deve confirmar: o índice FAISS, os casos de avaliação e a escala de maturidade.

2. Problema e domínio

Um comprador precisa de uma peça, em certa quantidade, às vezes com um orçamento e um prazo. Vários fornecedores a oferecem com preço, frete, estoque, prazo e pedido mínimo diferentes. O assistente deve dizer qual convém e por que as outras não.

O catálogo é uma base SQLite sintética e reproduzível (semente 20261107): 2.640 peças, 7.920 ofertas, 7.920 compatibilidades e 47.520 preços históricos. Cada conversa trabalha sobre um cenário isolado: uma cópia da base com um identificador próprio e um manifesto com a sua impressão digital. O assistente só lê: não compra nem paga.

3. O núcleo: comparar fornecedores

Todo o resto existe para que esta comparação seja correta e confiável.

Regras

Regra	A oferta é descartada se…
Estoque	Não há unidades suficientes
Pedido mínimo	O fornecedor exige comprar mais unidades do que as pedidas
Prazo	Demora mais que o prazo máximo do usuário, se ele informou
Orçamento	O total passa do orçamento do usuário, se ele informou

O total é quantidade vezes preço unitário mais frete, sempre em centavos inteiros. As ofertas são ordenadas assim: primeiro as que atendem; entre elas, as da origem que o usuário prefere (nacional ou internacional), se ele salvou essa preferência; depois por total, prazo e fornecedor. A preferência nunca faz ganhar uma oferta que não atende, e se respeitá-la custa mais, a resposta diz isso. Cada resposta mostra todas as ofertas, com a origem e o motivo do descarte.

A descoberta que mudou a comparação

Durante quase todo o projeto a comparação ignorou o pedido mínimo, embora o dado estivesse no catálogo. O exemplo de todas as demonstrações, 2 unidades de PEC-01-0001, recomendava a CentroPeças por R$ 2.963,07, e esse fornecedor exige 5 unidades. Há 1.151 ofertas com estoque e pedido mínimo maior que 2. Nenhum teste nem métrica detectou o problema: a referência da avaliação tinha a mesma lacuna, então o agente e a referência concordavam. Ele foi encontrado ao revisar quais dados do fornecedor a comparação usava. Com a correção, mudaram 5 dos 14 casos de recomendação da avaliação. Hoje as demonstrações usam 5 unidades de PEC-01-0001: CentroPeças, R$ 7.279,83.

Histórico e aviso de preço

O catálogo guarda seis preços mensais por oferta. Para cada oferta válida a resposta situa o preço de hoje: o do mês anterior, a média dos cinco meses anteriores e se está no ponto mais baixo ou mais alto. O histórico informa e não decide: ganha o menor total de hoje.

Quando o preço de hoje se afasta pelo menos 10 % da sua média, é acrescentado um aviso. Esse aviso reúne as técnicas do curso sobre o problema central:

Passo	Técnica	Rubrica
Sinal do preço, fatos e aviso fixo	Funções hardcoded	R19
Conselho redigido pelo modelo	Prompt PTCF com few-shot	R11
O conselho chega como objeto validado	Output parser Pydantic	R04
Aviso e conselho são calculados em dois ramos, com plano B	RunnableParallel e with_fallbacks	R06, R09, R12
O conselho só é publicado se os seus números saem dos fatos e o NLI o sustenta	Verificação exata e NLI	R24

Com o modelo e o NLI reais, sobre quinze peças: na versão anterior em espanhol, passaram na verificação 5 conselhos com o primeiro prompt (docs/rastros/precios_consejo_llm_antes.json) e 14 com o prompt ajustado. Na reexecução em português passaram de novo 14 de 15 (docs/rastros/precos_conselho_llm.json); o que ficou de fora citava um número que não estava nos fatos. Com o primeiro prompt, o modelo acrescentava avaliações («una buena oportunidad», citação da execução em espanhol) imitando um exemplo few-shot, e o NLI as rejeitava. É aqui, ao verificar texto livre, que o modelo NLI faz um trabalho que o cálculo exato não consegue fazer.

4. Arquitetura

Um turno segue sempre o mesmo percurso: interpretar a mensagem, decidir o caminho, consultar, verificar e responder.

- **Interpretar.** As regras identificam a peça, que tem formato fixo; um intérprete com modelo decide se a mensagem é uma compra e o que ela muda. Se a mensagem traz várias intenções, um planejador a divide em pedidos.
- **Decidir.** O que é reconhecido sem ambiguidade vai por uma rota fixa (listagem, ficha, esclarecimento). O resto vai para o agente.
- **Consultar.** O agente, construído com create_openai_tools_agent e AgentExecutor, escolhe entre seis ferramentas: três de SQL, uma de RAG, uma de API e uma de memória.
- **Verificar.** Toda proposta passa por uma verificação exata e por um modelo NLI. Antes, um passo de ancoragem preenche com dados das ferramentas o que o modelo copiaria errado.
- **Responder.** Só a pergunta e a resposta publicada são salvas na memória.

Um princípio atravessa o projeto: **o modelo decide o que responder; os dados quem põe é o sistema.** Valores, vencedor, citações, cidade de um CEP e tabelas de resultados saem sempre das ferramentas, nunca do que o modelo escreve.

A interface é Streamlit (app.py) e tem só o chat; toda a lógica fica em src/pedro/. Sem chave da OpenAI o chat funciona em modo degradado, com as rotas fixas e a comparação por regras.

5. Fase 1 — Chain executável e arquitetura (R01–R06)

Rubrica	O que foi feito	Resultado principal
R01	Anatomia, tipo e maturidade do agente	Agente híbrido; maturidade nível 4 demonstrada
R02	Planejamento e memória aumentada	Cinco passos observáveis e planejador de turno
R03	Modelos de prompt reutilizáveis	Nove ChatPromptTemplate com nome próprio
R04	Saída estruturada em JSON	Laboratório de oito saídas quebradas
R05	invoke, batch, stream e @chain	Os três medidos com o modelo real
R06	LCEL e justificativa	Os dois estilos comparados pelo que o rastro deixa ver

R01 — Anatomia, tipo e maturidade

A anatomia é lida dos objetos reais do agente, não é escrita à mão: modelo, executor, prompt PTCF, seis ferramentas, memória, base de conhecimento e porta de publicação. Pedro é um agente **híbrido**: uma camada reativa de regras resolve o que não deve ficar a critério do modelo, e uma camada deliberativa escolhe ferramentas, mantém estado e revisa a sua proposta. A maturidade fica no nível 4 de uma escala genérica de cinco (agente com memória e autoverificação) e está demonstrada com medições. Não se busca o nível 5: o problema não exige vários agentes nem executar compras. A escala é uma suposição declarada, porque a da primeira aula não está disponível.

R02 — Planejamento e memória aumentada

Uma solicitação é dividida em cinco passos com saída observável: extrair, consultar, comparar, verificar e publicar. Uma mensagem com várias intenções passa antes por um planejador de turno: o modelo escreve um plano com um pedido por intenção, o plano é validado (não pode acrescentar dados que o usuário não escreveu) e cada pedido é resolvido e verificado separadamente. Com o modelo real, uma mensagem que misturava compra, política e CEP gerou três pedidos, os três publicados (docs/rastros/r02_planejamento_llm.json). Antes do planejador, na versão anterior em espanhol, nada era publicado (docs/rastros/r02_planificacion_llm_sin_plan.json). A memória é dividida em dois níveis: a sessão guarda o que só serve para a compra em andamento e o usuário guarda o que deve sobreviver. Preços e estoque não são memória: são sempre relidos.

R03 — Modelos de prompt reutilizáveis

Os modelos de prompt estão reunidos em src/pedro/prompts.py, todos ChatPromptTemplate: extração (em três variantes), reparo, text-to-SQL, planejador, intérprete, conselho de preço e agente. O de extração é reutilizado na cadeia declarativa e na imperativa, e foi executado com o modelo real sobre três solicitações do domínio. Escolheu-se ChatPromptTemplate porque separa as regras (mensagem de sistema) do que o usuário escreve (mensagem humana).

R04 — Saída estruturada em JSON

A cadeia converte uma solicitação em um objeto validado com Pydantic. Para testar a robustez foi construído um laboratório com oito saídas típicas de um modelo; cada defeito é resolvido por uma camada diferente: o parser (cercas de markdown), uma segunda cadeia que repara (JSON truncado, tipos incorretos, texto livre) ou uma regra de domínio (os faltantes são calculados pelo código). Com o modelo real como reparador, os cinco reparos devolveram os dados corretos. O laboratório revelou que o parser aceita JSON truncado e perde um dado sem avisar; a cadeia robusta confere isso. Também foi executado with_structured_output: deu o mesmo resultado que o parser em 2 das 3 solicitações; na terceira, sem ID de peça, classificou a intenção e

os faltantes de outro jeito. Mantém-se o parser porque deixa ver e reparar a saída bruta.

R05 — Runnable e composição imperativa

Com o modelo real: invoke produz um objeto validado; batch processou três solicitações em 1,47 s, contra 4,83 s de três invoke em série; stream entregou 20 estados parciais, o primeiro aos 0,59 s. O que se aprendeu: stream só transmite se todos os passos sabem transmitir, e com o parser Pydantic chega um único fragmento. A composição imperativa usa @chain em duas cadeias; uma repete com try/except o reparo de R04 e dá o mesmo resultado no laboratório. São medições únicas, não um benchmark.

R06 — Composição declarativa e escolha

Há quatro cadeias LCEL (sequência, fallback, stream parcial e paralelo). A cadeia com reparo existe nos dois estilos e dá o mesmo resultado; o que muda é o que se observa: na versão declarativa o rastro registra 16 passos quando há reparo, incluído o erro, e na imperativa 9, sem o erro. Decisão: LCEL para o que é um encadeamento, e imperativo para decidir o caminho de cada turno, que tem ramos e saídas antecipadas.

6. Fase 2 — Agente com ferramentas e ReAct (R07–R12)

Rubrica	O que foi feito	Resultado principal
R07	Comparação de orquestradores	Seis comparados; LangChain e LangGraph executados
R08	ReAct com initialize_agent	Responde bem com memória e falha sem ela
R09	Chains e RouterChain	Router por regras 2 de 7; router LLM 6 de 7
R10	API externa por function calling	ViaCEP real em três casos
R11	PTCF, few-shot e chain-of-thought	Zero-shot 6 de 10; few-shot 10 de 10
R12	Componentes de confiabilidade	Os quatro, com caso normal e de erro

R07 — Orquestradores

São comparados LangChain, LangGraph, LlamaIndex, AutoGen, Semantic Kernel e Langflow por autonomia, abstração e modularidade. O mesmo agente pequeno foi executado no LangChain e no LangGraph: deram as mesmas respostas, mas o LangGraph guardou no seu estado o resultado de uma ferramenta e não a chamou de novo. Para o Pedro, em que preços e estoque devem ser relidos, essa diferença importa. O LangChain é mantido porque o projeto usa 13 tipos de componentes dele e porque a rubrica cita as suas APIs. Três delas estão marcadas como obsoletas; a migração natural seria para o LangGraph. Os outros quatro orquestradores são comparados de forma conceitual.

R08 — ReAct

Um agente com initialize_agent e ZERO_SHOT_REACT_DESCRIPTION, duas ferramentas e ConversationBufferMemory foi executado com o modelo real e deixou escrito o seu ciclo de pensamento, ação e observação. O que se aprendeu: o prompt zero-shot original não lê a memória. Com o histórico dentro do prompt resolveu uma pergunta de continuação (CentroPeças, 2 dias); sem ele, pediu de novo a peça.

R09 — Chains e routing

Há cadeias sequenciais e uma paralela, e dois routers sobre RouterChain. Em sete perguntas, o router por regras acertou 2 rotas e o decidido pelo modelo, 6. As perguntas foram escolhidas para pôr as regras à prova. O chat não usa nenhum deles como porta de entrada: o agente já roteia ao escolher a ferramenta.

R10 — API externa

O agente chama o ViaCEP, uma API pública real, por function calling. Estão salvos o esquema que o modelo recebe, os argumentos que ele envia, a resposta da API e o que foi publicado, em três casos: válido (São Paulo/SP), inexistente e inválido. A primeira execução, na versão anterior em espanhol, não publicou a cidade porque o modelo copiava errado um identificador (docs/rastros/r10_viacep_llm_antes.json); agora a resposta é preenchida com o que a ferramenta devolveu.

R11 — Prompts

O prompt do agente segue PTCF (Persona, Tarefa, Contexto, Formato) e leva exemplos few-shot. Foram medidas três técnicas em dez solicitações difíceis: sem exemplos 6 acertos, com few-shot 10 e com few-shot e chain-of-thought 10. O raciocínio é escrito em um campo antes dos dados, de modo que é observável. No agente, o pensamento passo a passo são as suas chamadas de ferramentas, que ficam registradas. O aviso de preço trouxe outra lição: o modelo imita o estilo do exemplo, não só o formato.

R12 — Confiabilidade

Os quatro componentes têm um caso normal e um de erro: parser com reparo, memória que guarda só o que foi publicado, busca FAISS com filtro e um callback que registra cada ferramenta e cada verificação. Esse registro é o que permitiu encontrar as falhas do agente com o modelo real.

7. Fase 3 — Chatbot com memória (R13–R18)

Rubrica	O que foi feito	Resultado principal
R13	Memórias segundo CoALA	As quatro, com o seu conteúdo após uma conversa real
R14	Janela e busca vetorial	Janela de 12 sobre 16 mensagens; três lembranças recuperadas
R15	Ferramentas de memória	LangMem executado frente à memória própria
R16	ConversationBufferMemory	Chatbot de quatro turnos que lembra tudo
R17	Intenção e entidades	Regras sozinhas 8 de 14; regras e modelo 14 de 14
R18	Conversa real multi-turno	Oito turnos com o estado correto

R13 — Memórias

Pedro tem as quatro memórias de CoALA: de trabalho (mensagem, rascunho de compra e janela, por sessão), episódica (comparações passadas com vetor, por usuário), semântica (preferências, catálogo e políticas) e procedimental (prompt e regras). Uma preferência é um fato e é aplicada: ordena as ofertas de cada comparação. Um episódio é um acontecimento e só é lembrado, porque os seus dados podem ter vencido.

R14 — Janela de contexto

O agente recebe uma janela deslizante das últimas 12 mensagens mais os três episódios mais parecidos com a pergunta. Depois de uma conversa real havia 16 mensagens salvas e 12 na janela; em uma sessão nova foram recuperados três episódios (similaridade entre 0,22 e 0,27) e nenhum para outro usuário. Escolheu-se janela e não resumo porque a janela não pode alterar um número.

R15 — Ferramentas de memória

São comparados Mem0, Letta/MemGPT e LangMem com a memória própria. O LangMem foi executado com a mesma conversa: as duas salvaram a preferência e isolaram por usuário. O LangMem deixa o modelo decidir o que lembrar e soube responder o que lembrava; o agente dele, sem verificação, deu uma resposta contraditória sobre estoque e prazo. Mantém-se a memória própria porque o que entra deve poder ser conferido. Mem0 e Letta são comparados de forma conceitual.

R16 — Chatbot com buffer

Um chatbot com ConversationChain e ConversationBufferMemory lembrou em quatro turnos o nome, a quantidade, o orçamento e o prazo. A memória cresceu de 313 para 954 caracteres: o buffer não tem limite, e por isso o agente final usa a janela.

R17 — Intenção e entidades

O chat reconhece seis intenções e cinco entidades. As regras identificam a peça e um intérprete few-shot decide se a mensagem é uma compra e o que ela muda. Em catorze mensagens difíceis, as regras sozinhas acertaram 8 e com o modelo 14: ele entende «Suba para meia dúzia», «Esqueça o prazo» ou «Dobre a quantidade». Quando falta a peça ou a quantidade, pergunta em vez de inventar.

R18 — Conversa real

Em oito turnos com o modelo real o agente descartou uma compra de 2 unidades por pedido mínimo, recalculou com 5 mantendo o prazo, trocou o prazo por um orçamento, salvou uma preferência, respondeu um CEP e uma política sem mexer na compra, voltou a comparar com o estado correto e aplicou a preferência a outra peça, em que o fornecedor nacional ganhou custando mais. A primeira versão desta conversa, na versão anterior em espanhol, falhou em três turnos (docs/rastros/fase3_memoria_llm_antes.json); os dois rastros estão guardados.

8. Fase 4 — SQL, APIs e RAG (R19–R23)

Rubrica	O que foi feito	Resultado principal
R19	Ferramentas hardcoded e semânticas	Seis ferramentas catalogadas
R20	APIs por tipo de uso	Três externas e quatro internas
R21	Text-to-query	5 de 5 perguntas; 7 de 7 consultas hostis rejeitadas
R22	Índice FAISS existente	O agente o carrega, não o constrói
R23	Falha de recuperação	Diagnosticada e corrigida, com antes e depois

R19 — Ferramentas

Seis ferramentas: quatro de lógica fixa, uma semântica (políticas) e uma mista (catálogo). A regra de divisão: tudo o que é um número ou uma regra é feito por código fixo; o semântico busca por significado e nunca decide um valor.

R20 — APIs

Três externas (ViaCEP para confirmar o destino, OpenAI como motor e LangSmith para avaliar) e quatro internas (catálogo em somente leitura, políticas em FAISS, memória isolada e verificação). O tipo de uso define a integração: o que lê é somente leitura, o que chama um terceiro tem limite de tempo e estados de erro, e a verificação fica fora do alcance do modelo. Nenhuma API compra nem modifica o catálogo.

R21 — Text-to-query

A ferramenta de catálogo tem dois modos. Com um ID de peça usa modelos fixos de consulta. Para perguntas abertas há um text-to-SQL completo: o modelo escreve um SELECT e o motor decide se o executa. Três barreiras: a forma (uma única instrução SELECT), o autorizador do SQLite (só as tabelas e colunas de uma lista) e os limites (somente leitura, 50 linhas). Com o modelo real respondeu bem 5 de 5 perguntas. Na versão anterior em espanhol, antes de melhorar o prompt, eram 3 de 5 (docs/rastros/fase4_sql_llm_antes.json): duas consultas eram válidas, mas estavam mal formuladas. Válida não significa correta, e por isso o chat mostra sempre a consulta executada. A tabela de resultados é montada pelo sistema.

R22 — Índice FAISS existente

O índice é construído uma vez, fora do agente, e salvo com um manifesto. O agente só o carrega, em 0,03 s. Como o seu armazenamento é um arquivo pickle, a carga exige um manifesto completo, impressões digitais que coincidam e confiança explícita; foi testado que ele rejeita um arquivo alterado. O índice é próprio: a atividade não entregou nenhum.

R23 — Falha de recuperação

A pergunta «Posso receber em várias caixas?» não recuperava a regra correta, que está no corpus. Diagnóstico: desajuste de vocabulário; o usuário diz «várias caixas» e a política diz «entrega parcial». Correção: reformular a consulta com o termo do domínio. Com o mesmo índice e os mesmos quatro resultados, o fragmento FIC-ENT-01-S05-C01 passou de não aparecer para a primeira posição (docs/rastros/r23_t24_indice_existente.json).

Esta pergunta foi medida de novo na migração. A tradução literal da pergunta espanhola, «Pode mandar em várias caixas?», deixou de falhar no corpus em português: o fragmento esperado ficou na posição 4 de 4. Foram medidas oito formulações naturais e escolhida uma que falha de verdade.

9. Fase 5 — Agente final verificado e avaliado (R24–R29)

Rubrica	O que foi feito	Resultado principal
R24	Verificação factual e lógica	Nenhuma recomendação contrária à regra publicada
R25	Agente único	SQL, RAG e API sob o mesmo executor
R26	create_openai_tools_agent e AgentExecutor	Executado em conversa, avaliação e interface
R27	RunnableWithMessageHistory	Só se salva o que foi publicado
R28	Streamlit	O app executado com o agente real
R29	LangSmith	Nove experimentos com seis métricas

R24 — Verificação

Nenhuma resposta é mostrada sem passar por duas portas dentro do fluxo. A verificação exata recalcula a resposta com os dados do cenário: total, estoque, pedido mínimo, orçamento, prazo, vencedor, identificadores e procedência. O modelo NLI classifica a relação entre evidência e afirmação em três classes, e só entailment permite publicar. Um passo anterior de ancoragem preenche com dados das ferramentas as citações, o CEP, as preferências, os números da comparação e as tabelas do catálogo. Se a verificação falha há uma correção; se falha de novo, não se publica. Limite importante: a verificação confere contra a regra de negócio, não confere a regra; enquanto a comparação ignorou o pedido mínimo, aprovou recomendações que um fornecedor não aceitaria.

R25 e R26 — Agente único e executor

Um único agente, construído com create_openai_tools_agent e AgentExecutor (seis iterações no máximo), reúne ferramentas de SQL, RAG e API. Na conversa de oito turnos usou as três categorias com o mesmo executor, a mesma memória e o mesmo verificador.

R27 — Histórico

O agente está envolvido em RunnableWithMessageHistory, com o histórico em SQLite por usuário e sessão. Só se salva o que foi publicado: depois de oito turnos havia 16 mensagens, oito do usuário e oito respostas. O histórico sobrevive à recriação do fluxo.

R28 — Interface

app.py conecta o Streamlit ao mesmo fluxo. Executado com a ferramenta de testes do Streamlit e o modelo real, respondeu quatro turnos de chat, abriu um chat novo e reabriu a conversa anterior com as suas 8 mensagens, sem erros. A interface é só o chat: o catálogo e a comparação são pedidos ali, com as ferramentas do agente. Cada resposta de compra é desenhada com dados: os passos que o agente seguiu, quatro cartões (fornecedor, total, entrega e origem), o conselho ou aviso de preço em destaque e a tabela de ofertas com a descartada e o motivo. Esses dados viajam com a mensagem já verificada, então a interface não relê o texto e, ao reabrir uma conversa, os cartões são redesenhados. Tudo usa componentes nativos do Streamlit, sem CSS; há duas capturas em docs/capturas/. No chat, a compra de 6 unidades de PEC-01-0001 deu R$ 8.718,75, o mesmo valor da demonstração de R02.

R29 — Avaliação com LangSmith

O fluxo final foi avaliado sobre 30 casos derivados com seis métricas. Resultado vigente, da execução em português (experimento pedro-infnet-fluxo-gpt-4.1-mini-portugues-235aa9f1, dataset pedro-infnet-casos-derivados-efff08ab, registro em docs/rastros/r29_langsmith_gpt-4.1-mini_portugues.json):

Métrica	Resultado
Vencedor correto	14 de 14
Valor correto	14 de 14
Conteúdo correto	16 de 16
Ferramenta decisiva	23 de 23
Primeira proposta aceita	24 de 24

Métrica	Resultado
Erros de verificação	0
Trajetória completa	26 de 30

Conclusões, cada uma com o seu dado. Os oito experimentos anteriores são da versão em espanhol do projeto e mantêm os seus números:

- **Mantém-se gpt-4.1-mini.** Em igualdade de condições, na versão em espanhol, gpt-4o-mini acertou 13 de 14 vencedores e 19 de 23 propostas na primeira tentativa.
- **A verificação cumpre o papel.** Quando um modelo propôs um vencedor errado, a proposta foi rejeitada e não foi publicada.
- **A avaliação encontrou um defeito do agente** (o CEP não era publicado) e **um erro de método**: os experimentos compartilhavam memória, e com 73 lembranças acumuladas a trajetória caiu de 25 para 15 de 30 (versão em espanhol).
- **A trajetória completa é uma métrica instável.** Variou entre 13 e 25 de 30 nas execuções da versão em espanhol e ficou em 26 de 30 na execução em português, porque a referência exige um passo que o agente às vezes pula com o mesmo resultado.
- **Uma métrica perfeita não garante uma regra correta.** Durante seis experimentos, na versão em espanhol, o agente obteve 14 de 14 recomendando ofertas que descumpriam o pedido mínimo, porque a referência compartilhava o erro.

10. O que as execuções reais ensinaram

Estas falhas só apareceram ao executar com o modelo real. Os testes automáticos já passavam. Elas foram observadas na versão anterior do projeto, em espanhol; na reexecução em português de 2026-11-11 as correções continuaram funcionando.

Falha observada	Causa	Correção
As perguntas de política não eram publicadas	O modelo citava o fragmento quase literal	O sistema ancora a citação no texto da fonte
A cidade de um CEP não era publicada	O modelo copiava errado um identificador	A resposta é preenchida com a observação da API
Uma compra válida não era publicada	O modelo transcrevia errado o vencedor	Os números quem põe é a ferramenta
Uma mensagem com várias intenções não era publicada	A resposta admite um só tipo	Planejador de turno
Não entendia «Suba para meia dúzia» nem «Esqueça o prazo»	Regras de diálogo estreitas demais	Intérprete com modelo
O parser perdia um dado com JSON truncado	Tolerância do parser	Conferência estrita
Duas consultas SQL válidas davam o resultado errado	O modelo confundia ID e SKU	Formato dos valores no prompt
A trajetória mudava entre experimentos	Memória compartilhada	Memória nova em cada experimento
Recomendava ofertas que o fornecedor não aceitaria	A comparação ignorava o pedido mínimo	Regra nova e casos regenerados
Os conselhos de preço eram rejeitados	Um exemplo few-shot convidava a avaliar	Exemplos que só enunciam fatos

11. Limitações

- **Suposições sem confirmar.** O índice FAISS é próprio, os 30 casos de avaliação são derivados e não os oficiais, e a escala de maturidade é genérica. O enunciado original em português não está disponível para uma conferência final.

- **Medições de uma única execução.** O modelo não é determinístico; as diferenças de um ou dois casos não são conclusivas.
- **Pedro não conversa.** Não responde «o que você lembra de mim?» nem sabe responder a um comentário sem pedido.
- **A verificação confere se a resposta é verdadeira, não se ela vem ao caso**, e confere contra a regra de negócio, não confere a regra.
- **O aviso de preço dispara muito** (em quatro de cada dez recomendações); os seus limiares não estão calibrados.
- **O modelo NLI não é exato.** Na versão em espanhol rejeitou um conselho que era verdadeiro; falha para o lado seguro. Na execução em português o mesmo caso foi barrado antes, pela verificação exata dos números. O NLI admite 512 tokens.
- **Comparações conceituais.** LlamaIndex, AutoGen, Semantic Kernel, Langflow, Mem0 e Letta não foram executados.
- **Sem credenciais** o chat interpreta só com regras, que são frágeis diante de formas de falar não previstas.
- **A comparação não divide um pedido entre fornecedores** nem calcula o frete conforme o destino.

12. Melhorias futuras

- Acrescentar um tipo de resposta conversacional, para saudações, comentários e «o que você lembra de mim?».
- Migrar a orquestração do turno para o LangGraph e manter os componentes do LangChain.
- Calibrar com dados os limiares do aviso de preço e da ancoragem de citações.
- Revisar a referência de trajetória com os casos oficiais.
- Usar o CEP para estimar frete e prazo conforme o destino.
- Resumir o histórico que sai da janela de 12 mensagens.

13. Como reproduzir as evidências

- Instalar: uv sync --python 3.13 --all-extras --locked
- Testes: uv run pytest -q
- Catálogo sintético: uv run python scripts/gerar_catalogo_pecas.py
- Índice de políticas: uv run python scripts/construir_indice_politicas.py
- Notebooks, sem credenciais: uv run python scripts/criar_notebooks.py e uv run python scripts/executar_notebooks.py
- Execuções com o modelo real (exigem OPENAI_API_KEY): os scripts scripts/demonstrar_*.py, que escrevem em docs/rastros/
- Avaliação (exige também LANGSMITH_API_KEY): uv run python evals/avaliar_fluxo.py --saida docs/rastros/r29_langsmith_gpt-4.1-mini_portugues.json --rotulo portugues
- Interface: uv run streamlit run app.py
- Este relatório em PDF: uv run python scripts/gerar_relatorio_pdf.py

Os cinco notebooks foram executados; eles usam duplos de teste e dizem isso em cada célula. Os rastros de docs/rastros/ com sufixo «antes» ou «sin_plan» e os oito experimentos do LangSmith anteriores ao de português são execuções da versão anterior do projeto, em espanhol. Não foram refeitos e ficam guardados como registro.

14. Pendências de entrega

- Gravar o vídeo de demonstração e verificar o link do Drive.
- Conferir as rubricas com o enunciado original em português.
- Confirmar com o professor o índice FAISS, os casos de avaliação e a escala de maturidade.